ServiceHub Backend: What Is in Place

In one sentence

This began as the starter structure for ServiceHub and now includes account, business, service, staff, scheduling, and appointment flows. A scaffold is the foundation of a project; the remaining feature modules are being built on it.

What works today

- The server starts with ‘npm run dev‘.
- ‘GET /api/v1/health‘ responds with ‘{ "status": "ok" }‘.
- ‘POST /api/v1/auth/register‘ creates a user, a business, and an owner membership in one transaction.
- ‘POST /api/v1/auth/login‘ checks the password and returns a 15-minute bearer access token.
- ‘GET /api/v1/auth/me‘ returns the signed-in user and their business memberships.
- Owners and admins can add existing accounts to a business with ADMIN, STAFF, or CUSTOMER roles.
- ‘GET /api/v1/businesses/current‘ returns the active business selected from a verified membership.
- Services can be listed, created, updated, and deactivated within the selected business.
- Staff records can be created, updated, deactivated, and assigned to services within the selected business.
- Managers can replace weekly staff schedules; schedule times use the business time zone.
- Customers can view public service catalogs and available time slots, then book appointments as guests or signed-in users.
- Business owners and staff can view tenant appointments; owners and admins can cancel or complete them.
- Signed-in customers can review their own completed appointments.
- Booking and management changes create in-app notifications and business-scoped audit records.
- Passwords are stored using Node.js scrypt hashes; registration and login have a stricter rate limit.
- The health endpoint has an automated test. The test suite has not yet been rerun after the auth changes.
- Environment settings are checked with Zod when the app starts.
- Requests are logged, common security headers are set, CORS and JSON parsing are configured, and API requests are rate limited.
- Errors go through one handler and use a consistent JSON shape: ‘{ "error": { "code": "...", "message": "..." } }‘.

How the folders are organized

- ‘src/app.js‘ assembles Express middleware and routes. ‘src/server.js‘ starts the listening server.
- ‘src/config/‘ validates settings. ‘src/middlewares/‘ includes authentication, tenant membership, RBAC, request validation, rate limiting, and error handling.
- ‘src/modules/auth/‘ implements registration, login, and current-user profile. ‘src/modules/businesses/‘ implements the current business route and membership lookup.
- ‘src/modules/services/‘, ‘src/modules/staff/‘, and ‘src/modules/schedules/‘ implement tenant-filtered management flows.
- ‘src/modules/appointments/‘ implements public booking, slot availability, tenant

appointment lists, and status changes.
- ‘src/modules/users/‘ implements profile and business membership management.
- ‘src/modules/reviews/‘, ‘src/modules/notifications/‘, and ‘src/modules/audit-logs/‘ implement completed-appointment reviews, private in-app notifications, and scoped change history.
- ‘src/modules/‘ also contains placeholder modules for dashboard and platform admin.
- ‘src/database/‘ contains the Prisma client setup and transaction helper. Prisma models and migrations cover users, businesses, memberships, services, staff, weekly schedules, and appointments.
- ‘src/lib/‘, ‘src/jobs/‘, and ‘src/realtime/‘ hold placeholders for logging, Redis/BullMQ jobs, and Socket.IO.
- ‘tests/‘ contains the health endpoint test. ‘docs/‘ describes architecture, API conventions, and tenancy guidance.

What is not built yet

Password reset, email verification, refresh tokens, and session revocation are not implemented. Dashboard and platform-admin features are not implemented. Redis, BullMQ, and Socket.IO are not connected yet.

Tenant design

The design uses one shared PostgreSQL database. The current tenant middleware checks an authenticated user’s membership and active business status before attaching ‘req.tenant‘. It accepts ‘X-Business-Id‘ for users with multiple memberships. Future tenant-owned tables should include ‘businessId‘, and each repository query should filter by that verified ID.

Run it locally

Requires Node.js 22 or newer. From the project folder:

1. Run ‘npm install‘.
2. Copy ‘.env.example‘ to ‘.env‘ and set ‘DATABASE_URL‘ and a private ‘JWT_ACCESS_SECRET‘.
3. Apply the initial migration with ‘npm run prisma:deploy‘.
4. Run ‘npm run dev‘.
5. Open ‘http://localhost:3000/api/v1/health‘ or call the auth routes with an API client.

PostgreSQL must be running for registration, login, profile, and business routes. Redis is only a configuration value for now; the app does not connect to it yet.

Suggested build order

1. Complete user and business management, including password reset and email verification.
2. Implement appointments and booking conflict handling.
3. Add dashboard reporting and platform administration.